



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 4
по дисциплине «Методы оптимизации»

СТАТИСТИЧЕСКИЕ МЕТОДЫ ПОИСКА

Бригада 4

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватель

ЛЕМЕШКО БОРИС ЮРЬЕВИЧ

Новосибирск, 2026

Содержание	
1 Цель работы.....	3
2 Задание.....	3
3 Исследования	4
3.1 Метод простого случайного поиска	4
3.1.1 Зависимость от ε	4
3.1.2 Зависимость от P	4
3.2 Алгоритм глобального поиска 1	5
3.3 Алгоритм глобального поиска 2	6
3.4 Алгоритм глобального поиска 3	7
4 Вывод	8
5 Программа	8
5.1 Main.cpp	8
5.2 Func.h	9
5.3 Func.cpp	9
5.4 RandomSearch.h.....	9
5.5 RandomSearch.cpp	10

1 Цель работы

Ознакомиться со статистическими методами поиска при решении задач нелинейного программирования. Изучить методы случайного поиска при определении глобального экстремума функции.

2 Задание

№	Вид работы
1	1.1. Разработать программу для решения задачи поиска глобального экстремума с использованием метода простого случайного поиска и трех алгоритмов глобального поиска. 1.2. Исследовать метод простого случайного поиска глобального экстремума при различных ε и P. Результат представить в таблице 1.3. Исследовать алгоритмы поиска глобального экстремума. Сравнить результаты поиска по количеству вычислений функции и найденной точке экстремума. Исследование провести при различных значениях числа попыток m.
2	П.1.3 повторить при 5 разных начальных значениях ГСЧ. Сделать выводы об устойчивости различных алгоритмов.

4 Вариант

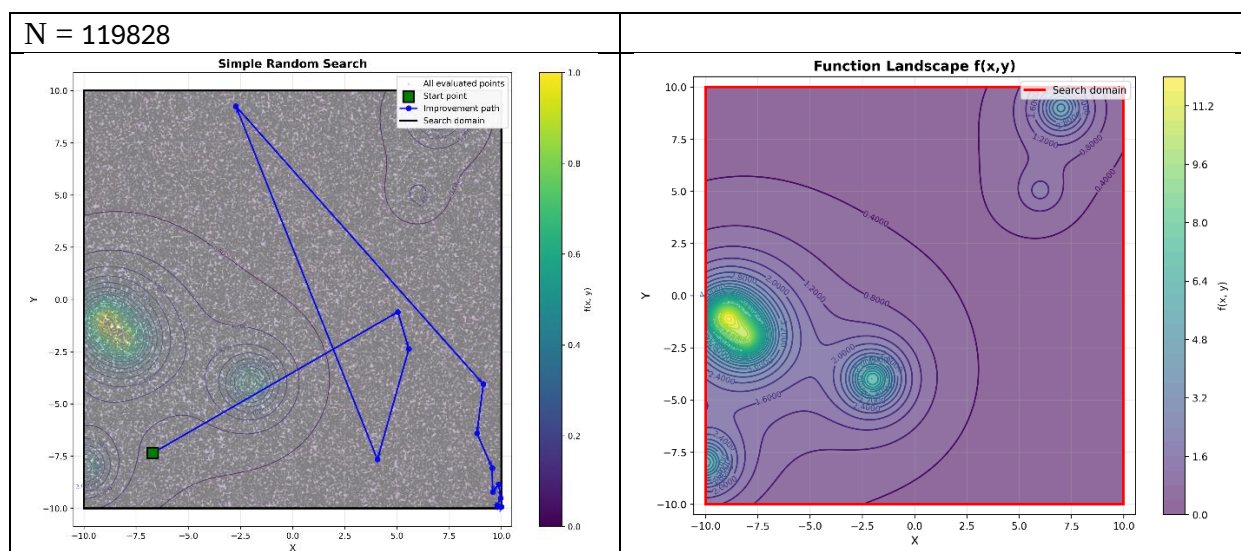
№	функция																	
4	$\sum_{i=1}^6 f(x,y) = \frac{C_i}{1 + (x - a_i)^2 + (y - b_i)^2}$																	
C_1	C_2	C_3	C_4	C_5	C_6	a_1	a_2	a_3	a_4	a_5	a_6	b_1	b_2	b_3	b_4	b_5	b_6	
4	9	1	7	5	6	7	-9	6	-8	-10	-2	9	-1	5	-2	-8	-4	

3 Исследования

3.1 Метод простого случайного поиска

3.1.1 Зависимость от ε

ε	P	N	$(x^*; y^*)$	$f(x^*; y^*)$
0.1	0.95	119828	(9.94461; -9.95546)	0.0994862
0.01		11982928	(9.99878; -9.99892)	0.098737
0.001		1198292915	(9.99983; -9.99983)	0.0987222



Чем выше точность решения, тем больше необходимо вычислений.

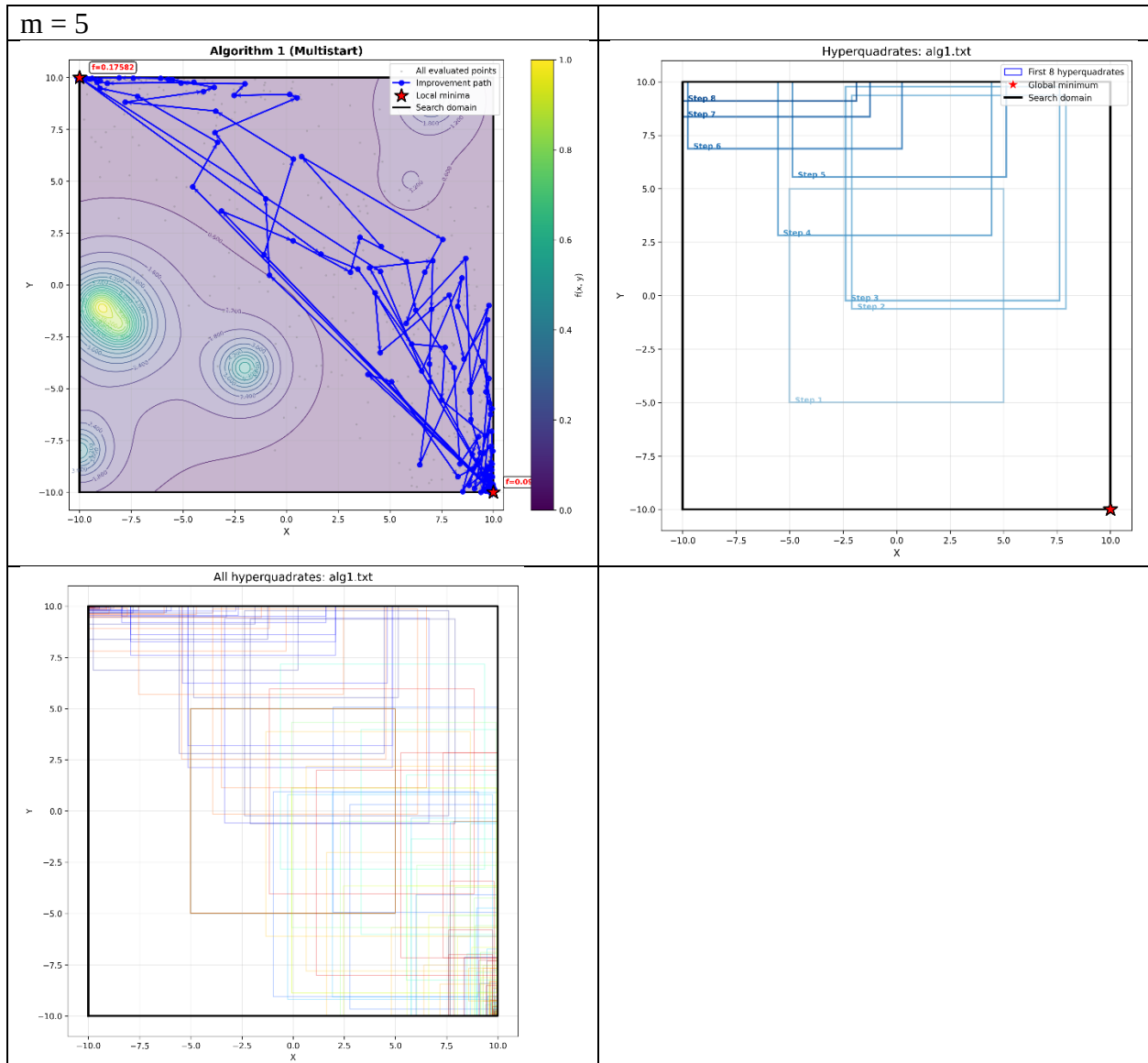
3.1.2 Зависимость от P

ε	P	N	$(x^*; y^*)$	$f(x^*; y^*)$
0.01	0.8	64377	(9.87611; -9.96177)	0.100101
	0.9	92103	(9.93351; -9.98319)	0.0994368
	0.95	119828	(9.99878; -9.99892)	0.098737
	0.99	184205	(9.96339; -9.98246)	0.0991594
	0.999	276307	(9.96823; -9.9781)	0.099138

Оптимальный параметр $P = 0.95$ даёт лучший баланс количества вычислений и точности. Для достижения высокой точности требуется огромное количество вычислений, что делает метод неэффективным для точного поиска

3.2 Алгоритм глобального поиска 1

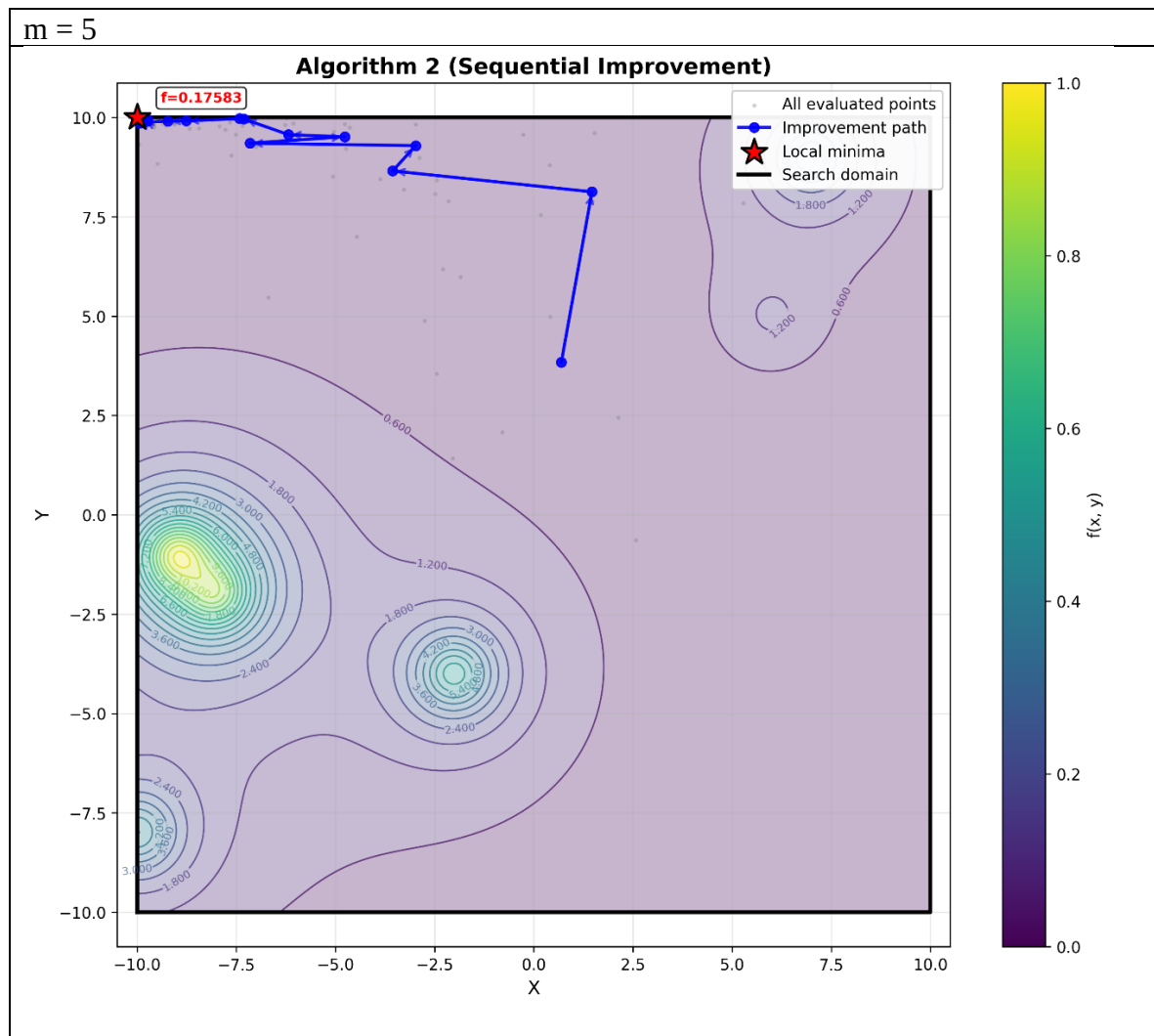
m	Вызовов $f(x; y)$	x^*	y^*	$f(x^*; y^*)$
5	1510	10	-9.99996	0.0987199
20	6010	10	-10	0.0987196
50	15010	10	-10	0.0987196



Точность результата растёт с повышением m .
(использован алгоритм с использованием гиперквadrата)

3.3 Алгоритм глобального поиска 2

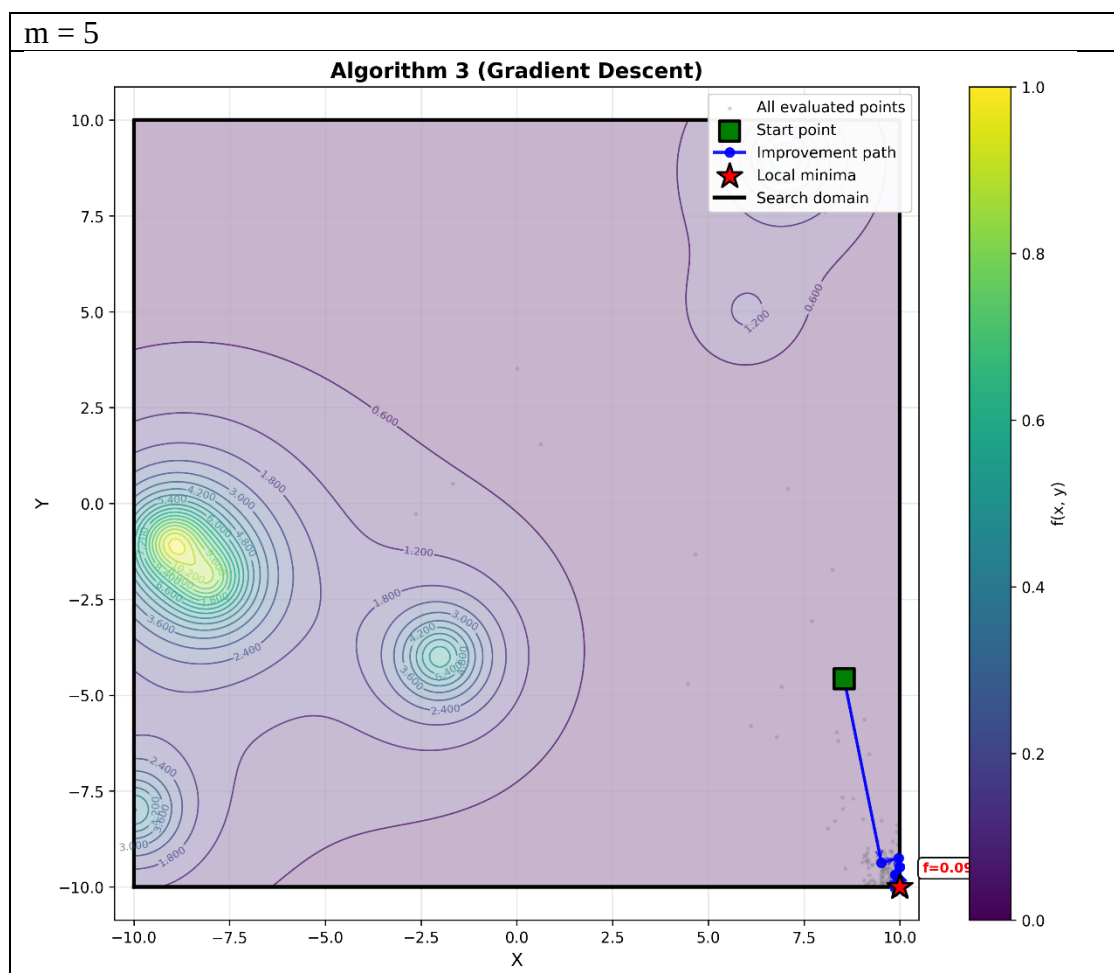
m	Вызовов $f(x; y)$	x^*	y^*	$f(x^*; y^*)$
5	156	-9.99881	9.99999	0.0987221
20	1240	10	-10	0.0987196
50	16854	10	-10	0.0987196



Метод эффективен при малых m.

3.4 Алгоритм глобального поиска 3

m	ВЫЗОВОВ $f(x; y)$	x^*	y^*	$f(x^*; y^*)$
5	1663	10	-10	0.0987196
20	6614	10	-10	0.0987196
50	16514	10	-10	0.0987196



4 Вывод

Все алгоритмы успешно находят глобальный экстремум (с высокой вероятностью) $(x^*; y^*) = (10; -10)$, $f(x^*; y^*) = 0.0987196$

Второй алгоритм глобального поиска находит результат быстрее всего, за счёт использования случайного поиска и локального спуска.

Третий алгоритм обеспечивает направленный выход из локальных минимумов, но требует больше вычислений из-за дополнительного поиска вдоль направления

При больших вычислениях ($m > 50$) разница в алгоритмах минимальна

Алгоритм 2 является **наиболее эффективным** для практического использования в задачах с ограниченными вычислительными ресурсами, превосходя 1 алгоритм по скорости в **5-10 раз** при малых затратах.

5 Программа

5.1 Main.cpp

```
#include <iostream>
#include "RandomSearch.h"

int main()
{
    SimpleRandomSearch method;
    method.input();
    int MethodType = 0;
    std::cout << "Enter method(1 - Simple search; 2 - Method hyperquadrate; 3 - algorithm_1; 4 - algorithm_2; 5 - algorithm_3):\n";
    std::cin >> MethodType;

    if (MethodType == 1)
    {
        method.log_file.open("simple.txt");
        method.Search();
        std::cout << "Simple search:\n";
        std::cout << "min f(x,y): " << method.func_0 << '\n';
        std::cout << "min x: " << method.x[0] << '\n' << "min y: " << method.x[1] << '\n';
    }
    else if (MethodType == 2)
    {
        method.hyperquadrate();
        std::cout << "Method hyperquadrate:\n";
        std::cout << "min f(x,y): " << method.func_0 << '\n';
        std::cout << "min x: " << method.x[0] << '\n' << "min y: " << method.x[1] << '\n';
        std::cout << "Count_use_function: " << method.func.count_use_f << '\n';
    }
    else if (MethodType == 3)
    {
        method.log_file.open("alg1.txt");
        method.algorithm_1();
        method.log_file.close();

        std::cout << "algorithm_1:\n";
        std::cout << "min f(x,y): " << method.func_0 << '\n';
        std::cout << "min x: " << method.x[0] << '\n' << "min y: " << method.x[1] << '\n';
        std::cout << "Count_use_function: " << method.func.count_use_f << '\n';
    }
    else if (MethodType == 4)
    {
        method.log_file.open("alg2.txt");
        method.algorithm_2();
        method.log_file.close();
    }
}
```



```

std::cout << "algorithm_2:\n";
std::cout << "min f(x,y): " << method.func_0 << '\n';
std::cout << "min x: " << method.x[0] << '\n' << "min y: " << method.x[1] << '\n';
std::cout << "Count_use_function: " << method.func.count_use_f << '\n';
}
else if (MethodType == 5)
{
method.log_file.open("alg3.txt");
method.algorithm_3();
method.log_file.close();

std::cout << "algorithm_3:\n";
std::cout << "min f(x,y): " << method.func_0 << '\n';
std::cout << "min x: " << method.x[0] << '\n' << "min y: " << method.x[1] << '\n';
std::cout << "Count_use_function: " << method.func.count_use_f << '\n';
}
else
{
std::cout << "Not correct method:\n";
return 0;
}
}

```

5.2 Func.h

```

#pragma once
#include <vector>

class Function
{
private:
std::vector<double> C = { 4, 9, 1, 7, 5, 6 };
std::vector<double> a = { 7, -9, 6, -8, -10, -2 };
std::vector<double> b = { 9, -1, 5, -2, -8, -4 };

public:
double f(double x, double y);
int count_use_f = 0;
};

```

5.3 Func.cpp

```

#include "Func.h"

double Function::f(double x, double y)
{
count_use_f++;
double sum = 0;

for (int i = 0; i < 6; i++)
{
sum += C[i] / (1 + (x - a[i]) * (x - a[i]) + (y - b[i]) * (y - b[i]));
}
return sum;
}

```

5.4 RandomSearch.h

```

#pragma once

#include "Func.h"

```

```

#include <fstream>
#include <random>
#include <vector>
#include <string>

class SimpleRandomSearch
{
private:

double x_a;
double x_b;
double y_a;
double y_b;
double eps;
int N_simple;
int N;
int m;
int m_global;

std::random_device rd;
std::mt19937 gen;

std::vector<double> a;
std::vector<double> b;
std::vector<double> a_start;
std::vector<double> b_start;

public:
double func_0;
std::vector<double> x;
Function func;
std::ofstream log_file;

public:
void input();
double getRandomDouble(double min, double max);
void Search(double x_min, double x_max, double y_min, double y_max, bool Ifbreak,
bool IfStartPoint);
void Search();
SimpleRandomSearch();
void hyperquadrate();
void hyperquadrate(std::vector<double> a, std::vector<double> b, bool
NeedStartPoint);
void algorithm_1();
void algorithm_2();
void algorithm_3();
void logPoint(const std::string& type, double x, double y);
void logQuadrate(const std::string& type, double x_min, double x_max, double y_min,
double y_max);
void logAllPoint(const std::string& type, double x, double y);
};

```

5.5 RandomSearch.cpp

```

#include "RandomSearch.h"

SimpleRandomSearch::SimpleRandomSearch() :
gen(rd())
{
x_a = 0;
x_b = 0;
y_a = 0;
y_b = 0;
eps = 1e-3;

```

```

N_simple = 0;
N = 0;
m = 2;
m_global = 1;
func_0 = 0;
x = { 0, 0 };
a = { 0, 0 };
b = { 0, 0 };
a_start = { 0, 0 };
b_start = { 0, 0 };
}

void SimpleRandomSearch::input()
{
    std::fstream in("in.txt");
    if (in.is_open())
    {
        in >> x_a;
        in >> x_b;
        in >> y_a;
        in >> y_b;
        in >> eps;
        in >> N_simple;
        in >> N;
        in >> m;
        in >> m_global;
        in >> a[0];
        in >> b[0];
        in >> a[1];
        in >> b[1];
        a_start[0] = a[0];
        b_start[0] = b[0];
        a_start[1] = a[1];
        b_start[1] = b[1];
    }
}

double SimpleRandomSearch::getRandomDouble(double min, double max)
{
    std::uniform_real_distribution<double> dist(min, max);
    return dist(gen);
}

void SimpleRandomSearch::Search(double x_min, double x_max, double y_min, double
y_max, bool Ifbreak, bool IfStartPoint)
{
    double func_1 = 0;

    if (IfStartPoint)
    {
        x[0] = getRandomDouble(x_min, x_max);
        x[1] = getRandomDouble(y_min, y_max);

        logPoint("S", x[0], x[1]);
    }
    func_0 = func.f(x[0], x[1]);

    for (int i = 0; i < N_simple; i++)
    {
        double x_1 = getRandomDouble(x_min, x_max);
        double y_1 = getRandomDouble(y_min, y_max);
        func_1 = func.f(x_1, y_1);

        logAllPoint("P", x_1, y_1);
    }
}

```

```

logPoint("R", x_1, y_1);

if (func_1 < func_0)
{
x[0] = x_1;
x[1] = y_1;
func_0 = func_1;

logPoint("B", x[0], x[1]);

if (Ifbreak) break;
}
}
}

void SimpleRandomSearch::Search()
{
Search(x_a, x_b, y_a, y_b, false, true);
}

void SimpleRandomSearch::hyperquadrate(std::vector<double> a, std::vector<double>
b, bool NeedStartPoint)
{
double func_1 = 0;

if (NeedStartPoint)
{
x[0] = getRandomDouble(a[0], b[0]);
x[1] = getRandomDouble(a[1], b[1]);
}
func_0 = func.f(x[0], x[1]);

for (int k = 0; k < N; k++)
{
logQuadrate("Q", a[0], b[0], a[1], b[1]);
for (int i = 0; i < m; i++)
{
double x_1 = getRandomDouble(a[0], b[0]);
double y_1 = getRandomDouble(a[1], b[1]);
func_1 = func.f(x_1, y_1);

logAllPoint("P", x_1, y_1);

if (func_1 < func_0)
{
x[0] = x_1;
x[1] = y_1;
func_0 = func_1;

logPoint("B", x[0], x[1]);
}
}

double a_i = 0;
double b_i = 0;

for (int i = 0; i < 2; i++)
{
a_i = a[i];
b_i = b[i];
a[i] = x[i] - (b_i - a_i) / 2;
b[i] = x[i] + (b_i - a_i) / 2;

if (i == 0)
{

```

```

    if (a[i] < x_a) a[i] = x_a;
    if (b[i] > x_b) b[i] = x_b;
    }
    else
    {
        if (a[i] < y_a) a[i] = y_a;
        if (b[i] > y_b) b[i] = y_b;
    }
    }
    }
    logPoint("L", x[0], x[1]);
    }

    void SimpleRandomSearch::hyperquadrate()
    {
        hyperquadrate(a_start, b_start, true);
    }

    void SimpleRandomSearch::algorithm_1()
    {
        double func_temp = 1e10;
        double x_temp = 0;
        double y_temp = 0;

        for (int i = 0; i < m_global; i++)
        {
            hyperquadrate(a_start, b_start, true);

            if (func_0 < func_temp)
            {
                func_temp = func_0;
                x_temp = x[0];
                y_temp = x[1];

                logPoint("B", x[0], x[1]);
            }
        }

        func_0 = func_temp;
        x[0] = x_temp;
        x[1] = y_temp;

        logPoint("L", x[0], x[1]);
    }

    void SimpleRandomSearch::algorithm_2()
    {
        hyperquadrate(a_start, b_start, true);

        double best_f = func_0;
        std::vector<double> best_x = x;

        for (int k = 0; k < m_global; k++)
        {
            bool found = false;
            double radius = 1.0;

            for (int attempt = 0; attempt < m; attempt++)
            {
                double x_rand = getRandomDouble(
                    std::max(x[0] - radius, x_a),
                    std::min(x[0] + radius, x_b)
                );

```

```

double y_rand = getRandomDouble(
std::max(x[1] - radius, y_a),
std::min(x[1] + radius, y_b)
);

double f_rand = func.f(x_rand, y_rand);

logPoint("R", x_rand, y_rand);

if (f_rand < func_0)
{
x[0] = x_rand;
x[1] = y_rand;
func_0 = f_rand;

logPoint("B", x[0], x[1]);

found = true;
break;
}

radius *= 0.5;
}

if (!found)
break;

std::vector<double> a_local =
{
std::max(x[0] - 1.0, x_a),
std::max(x[1] - 1.0, y_a)
};
std::vector<double> b_local =
{
std::min(x[0] + 1.0, x_b),
std::min(x[1] + 1.0, y_b)
};

hyperquadrate(a_local, b_local, false);

if (func_0 < best_f)
{
best_f = func_0;
best_x = x;
}
}

x = best_x;
func_0 = best_f;
}

void SimpleRandomSearch::algorithm_3()
{
x[0] = getRandomDouble(x_a, x_b);
x[1] = getRandomDouble(y_a, y_b);

logPoint("S", x[0], x[1]);

hyperquadrate(a_start, b_start, false);

double best_f = func_0;
std::vector<double> best_x = x;

for (int k = 0; k < m_global; k++)
{

```

```

std::vector<double> x0 = x;

double dx = getRandomDouble(-1.0, 1.0);
double dy = getRandomDouble(-1.0, 1.0);

double step = 0.2;

std::vector<double> x_new = x;
double f_prev = func_0;

while (true)
{
    std::vector<double> candidate =
    {
        x_new[0] + step * dx,
        x_new[1] + step * dy
    };

    if (candidate[0] < x_a || candidate[0] > x_b ||
        candidate[1] < y_a || candidate[1] > y_b)
        break;

    double f_new = func.f(candidate[0], candidate[1]);

    logPoint("R", candidate[0], candidate[1]);

    if (f_new < f_prev)
    {
        x_new = candidate;
        f_prev = f_new;

        logPoint("B", x_new[0], x_new[1]);
    }
    else
    {
        break;
    }
}

x = x_new;

std::vector<double> a_local =
{
    std::max(x[0] - 1.0, x_a),
    std::max(x[1] - 1.0, y_a)
};
std::vector<double> b_local =
{
    std::min(x[0] + 1.0, x_b),
    std::min(x[1] + 1.0, y_b)
};

hyperquadrate(a_local, b_local, false);

if (func_0 < best_f)
{
    best_f = func_0;
    best_x = x;
}
else
{
    x = best_x;
    func_0 = best_f;
}

```

```

    }
    }

    x = best_x;
    func_0 = best_f;
    }

    void SimpleRandomSearch::logPoint(const std::string& type, double x, double y)
    {
        if (log_file.is_open())
            log_file << type << " " << x << " " << y << "\n";
    }

    void SimpleRandomSearch::logQuadrante(const std::string& type, double x_min, double
x_max, double y_min, double y_max)
    {
        if (log_file.is_open())
        {
            log_file << type << " " << x_min << " " << x_max << " " << y_min << " " << y_max
<< "\n";
        }
    }

    void SimpleRandomSearch::logAllPoint(const std::string& type, double x, double y)
    {
        if (log_file.is_open())
            log_file << type << " " << x << " " << y << "\n";
    }

```